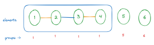
Disjoint Set Union (DSU)

Helps to perform grouping operation

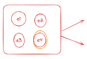


- union -> mechanism using which we can group elements together
- find -> mechanism using which we can find which group an element belongs to

union(1,2) -> we want to group 1 and 2 together

union(3,4). union(2,3)

find(4) -> 1. find(5) -> 5 find(1) -> 1 find(3) -> 1



We give the group a unique id

Make one element from the group as leader/parent and this element will be the representative of the group

How to build DSU

We need two main functions to build DSU:
- union
- find

0	3	1	3	3	5	6	7	8	9	DSU array
0.	1.	2.	3.	4.	5	6	7	8.	9	index

index -> element
arr[index] -> parent of the group where the element belongs

find(2) -> 2

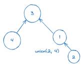
```
function find(x) {
  return arr[x]; // O(1)
}
```

union(1,2) union(3,4) union(2,4)

new parent here will be either the parent of 2 or parent of 4 -> randomly I picked 3 to be the parent

```
function union(x, y) { // O(n)
     px = find(x)
     py = fund(y)

     // final parent is px

     for(i = 0; i < n; i++) {
         if (arr[i] == py) arr[i] = px;
     }

}
```

all the elements of py now will be part of px, so wherever in the dsu array u find parent to be py update all of those parents to px



```
function union(x, y) { //O(n)

     px = find(x)
     py = find(y)

     arr[px] = py; // make the parent of px as py


}

function find(x) { // O(n)

     if(arr[x] == x) return x;
     return find(arr[x]);


}
```

find(2) -> find(1)-> find(3)

Union By Size

Don;t randomly assign the parent, always keep the parent of the group which is bigger in size

0	3	1	3	3	5	6	7	8	9	DSU array	parent array
0.	1.	2.	3.	4.	5	6	7	8.	9	index	
1	2	1.	5.	1.	1.	1.	1.	1.	1	size	

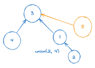
union(1, 2)

union(3, 4)

union(1, 3)

union(1, 5)

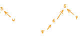
submerge the smaller in bigger group



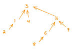
```
function union(x, y) { // O(logn)

     px = find(x)
     py = find(y)

     sx = size[px]
     sy = size[py]

     if(sx > sy ) {
         arr[py] = px; // make px the leader
         size[px] += size[py]
     } else {
         arr[px] = py;
         size[py] += size[px]

     }


}
function find(x) { // O(logn)
     if(arr[x] == x) return x;
     return find(arr[x]);

}
```

max ops - 2

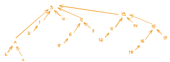
union(3, 5)



max operations to find parent of any element is just 3



max operation needed to find parent of any elemen t is 4



max ops - 5

Approx how much the max ops look like ?

approx log(size) + 1

The maximum number of operations with this approch to find parent of any element is

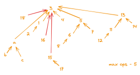
log(size) + 1

Worst case: O(log(n))

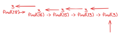
Union by rank

```
function union(x, y) { // O(logn)

     px = find(x)
     py = find(y)

     sx = rank[px]
     sy = rank[py]

     if(sx > sy ) {
         arr[py] = px; // make px the leader
         rank[px] += 1
     } else {
         arr[px] = py;
         rank[py] += 1

     }


}
```

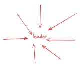
Path compression



max ops - 5



Base case



```
function find(x) {
     if(arr[x] == x) return x;

     arr[x] = find(arr[x]);

     return result;

}
```

O(log*n)

log*n computes the number of times we have to do logN to convert N to 1

N -> 64 -> logn -> 6

log*n -> no of times we have to do logn to convert n to 1

n -> 64 -> n = logn -> n = log64 -> n = 6

n-> 6 -> n = logn -> n = log6 -> n = 2.5

n -> 2.5 n = logn -> n = log2.5 -> n = 1.3

n -> 1.3 n = logn -> n = log1.3 -> n = approx 1

Ex: 2^ (65536) log (2^65536) -> 65536

log*(2^65536) -> 5

log(2^65535) -> 65536

log(65536) -> 16

log(16) -> 4

log(4) -> 2

log(2) -> 1